

Abstraksi dan Struktur Data Dinamis: Membangun Arsitektur Kode Fleksibel dengan Map dan Interface

Durasi: 3 Jam (180 Menit) **Level:** Intermediate (Menengah)

Menulis kode backend yang berkinerja tinggi tidak hanya tentang logika algoritma, tetapi juga tentang bagaimana merancang ketergantungan antar-komponen program. Jika komponen kode Anda terikat terlalu kaku (*tight coupling*), maka aplikasi akan sangat sulit diuji (*untestable*) dan dikembangkan di masa mendatang.

Modul ini akan membahas dua pilar krusial di Go untuk merancang sistem yang fleksibel dan mudah diperluas: **Map** untuk penyimpanan data asosiatif dinamis (*key-value*), dan **Interface** untuk menerapkan konsep abstraksi (*loose coupling*) serta polimorfisme standar industri.

Sesi 1: Struktur Data Dinamis: Map Deep Dive (45 Menit)

`map` adalah tipe data bawaan Go untuk menyimpan kumpulan data dengan hubungan kunci dan nilai (*key-value pairs*). Di balik layar, map diimplementasikan sebagai struktur *hash table* yang dinamis.

1. Deklarasi dan Inisialisasi Map

Sama seperti slice, map adalah **tipe referensi** (*reference type*). Nilai bawaan (Zero Value) dari map yang belum diinisialisasi adalah `nil`. Menulis data ke map `nil` akan menyebabkan crash (**panic**).

```

package main

import "fmt"

func main() {
    // Cara 1: Menggunakan make (Sangat direkomendasikan)
    productPrices := make(map[string]float64)
    productPrices["Laptop"] = 15000000.0
    productPrices["Mouse"] = 350000.0

    // Cara 2: Menggunakan map literal
    categories := map[string]string{
        "R0G": "Laptop Gaming",
        "MX3": "Mouse Productivity",
    }

    fmt.Println(productPrices["Laptop"])
    fmt.Println(categories["R0G"])
}

```

2. Memeriksa Keberadaan Kunci (*Comma OK Idiom*)

Jika Anda mencari kunci yang tidak terdaftar di dalam map, Go tidak akan melempar error, melainkan mengembalikan *zero value* dari tipe data nilai terkait. Hal ini bisa memicu ambiguitas (apakah harganya memang Rp 0, atau barangnya tidak terdaftar?).

Untuk mengatasinya, gunakan pola **comma ok**:

```

price, exists := productPrices["Keyboard"]
if !exists {
    fmt.Println("Barang tidak ditemukan!")
} else {
    fmt.Printf("Harga: Rp %.2f\n", price)
}

```

3. Menghapus Kunci & Iterasi Map

- **Menghapus Kunci:** Gunakan fungsi bawaan `delete(mapName, key)`.
- **Iterasi Map:** Gunakan perulangan `for range`.

```

// Menghapus data
delete(productPrices, "Mouse")

// Iterasi Map
for key, value := range productPrices {
    fmt.Printf("Produk: %s → Harga: Rp %.2f\n", key, value)
}

```

IMPORTANT

Penting Mengenai Iterasi Map: Urutan pengambilan elemen saat mengiterasi map menggunakan `for range` di Go adalah **acak (random/non-deterministic)**. Go sengaja mengacak urutan ini di setiap eksekusi agar pengembang tidak membuat asumsi logika berdasarkan urutan data map.

Sesi 2: Abstraksi Melalui Interface (45 Menit)

`interface` di Go adalah tipe data kustom yang mendefinisikan sekumpulan tanda tangan metode (*method signatures*), namun tidak menyediakan implementasinya sama sekali.

1. Implementasi Implisit (*Duck Typing*)

Di Go, Anda tidak perlu menulis keyword `implements` untuk menghubungkan sebuah struct dengan interface. Jika suatu struct memiliki **semua** metode dengan nama, parameter, dan nilai balik yang persis didefinisikan oleh sebuah interface, maka struct tersebut otomatis mengimplementasikannya secara implisit.

"Jika ia berjalan seperti bebek, berbunyi seperti bebek, maka ia adalah bebek."

2. Studi Kasus Abstraksi Pembayaran

```
package main

import "fmt"

// 1. Definisikan Interface kontrak pembayaran
type PaymentGateway interface {
    Pay(amount float64) error
}

// 2. Struct BankTransfer mengimplementasikan PaymentGateway secara implisit
type BankTransfer struct {
    BankName string
}

func (b BankTransfer) Pay(amount float64) error {
    fmt.Printf("Membayar Rp %.2f lewat transfer Bank %s\n", amount, b.BankName)
    return nil
}

// 3. Struct EWallet mengimplementasikan PaymentGateway secara implisit
type EWallet struct {
    Provider string
}

func (e EWallet) Pay(amount float64) error {
    fmt.Printf("Membayar Rp %.2f menggunakan E-Wallet %s\n", amount, e.Provider)
    return nil
}

// 4. Fungsi polimorfisme yang menerima interface apa pun yang cocok
func CheckoutOrder(gateway PaymentGateway, total float64) {
    err := gateway.Pay(total)
    if err != nil {
        fmt.Println("Pembayaran gagal:", err)
        return
    }
    fmt.Println("Transaksi Sukses!")
}

func main() {
    paymentMethod1 := BankTransfer{BankName: "BCA"}
    paymentMethod2 := EWallet{Provider: "GoPay"}

    // Fungsi yang sama bisa menerima dua tipe data konkret yang berbeda
    CheckoutOrder(paymentMethod1, 500000)
    CheckoutOrder(paymentMethod2, 120000)
}
```

Sesi 3: Empty Interface (`interface{}` / `any`) & Type Assertion

(45 Menit)

Sejak Go 1.18, terdapat alias baru bernama `any` untuk menggantikan interface kosong (`interface{}`).

1. Apa itu Empty Interface?

Interface kosong tidak memiliki daftar metode sama sekali. Karena tidak ada syarat kontrak metode, maka **semua tipe data di Go** (int, string, float, struct, slice, dll.) otomatis memenuhi kriteria ini.

```
var data any // atau var data interface{}

data = "Halo"
data = 42
data = true
```

2. Type Assertion & Type Switch

Ketika kita menampung data di dalam tipe `any`, kita kehilangan informasi tipe data konkretnya. Kita tidak bisa langsung menjumlahkan `data + 10` jika data bertipe `any`, meskipun saat itu isinya adalah angka. Kita harus mengembalikan tipe datanya terlebih dahulu.

A. Type Assertion

```
val, ok := data.(int) // Mencoba mengonversi data kembali ke int
if ok {
    result := val + 10
    fmt.Println("Hasil:", result)
} else {
    fmt.Println("Data bukan integer!")
}
```

B. Type Switch (Cara Lebih Rapi untuk Multi-tipe)

```
package main

import "fmt"

func ProcessData(d any) {
    switch v := d.(type) {
    case string:
        fmt.Printf("Data adalah string: %s (panjang %d)\n", v, len(v))
    case int:
        fmt.Printf("Data adalah int: %d (dikali dua = %d)\n", v, v*2)
    default:
        fmt.Println("Tipe data tidak didukung")
    }
}

func main() {
    ProcessData("Golang")
    ProcessData(100)
}
```

Sesi 4: Praktikum Mandiri: "Sistem Multi-Payment Gateway & Cart Cache" (45 Menit)

Mari kita gabungkan penggunaan **Map** sebagai in-memory cache belanja dengan **Interface** untuk menangani transaksi checkout menggunakan multi-metode pembayaran yang berbeda.

Implementasi Kode (`main.go`)

```
package main

import (
    "errors"
    "fmt"
)

// 1. Interface Kontrak Pembayaran
type PaymentProcessor interface {
    Pay(amount float64) error
    GetMethodName() string
}

// 2. Concrete Struct 1: Midtrans CC
type CreditCard struct {
    CardNumber string
}

func (cc CreditCard) GetMethodName() string {
    return "Kartu Kredit (Midtrans)"
}

func (cc CreditCard) Pay(amount float64) error {
    if len(cc.CardNumber) < 16 {
        return errors.New("nomor kartu kredit tidak valid")
    }
    fmt.Printf("[PROSES] Memotong limit kartu kredit %s sebesar Rp %.2f\n", cc.CardNumber, amount)
    return nil
}

// 3. Concrete Struct 2: ShopeePay Qris
type QrisPayment struct {
    QrisID string
}

func (q QrisPayment) GetMethodName() string {
    return "QRIS (ShopeePay)"
}

func (q QrisPayment) Pay(amount float64) error {
    if q.QrisID == "" {
        return errors.New("QRIS ID merchant kosong")
    }
    fmt.Printf("[PROSES] Menghasilkan QR Code dinamis untuk ID %s nominal Rp %.2f\n", q.QrisID, amount)
    return nil
}

// 4. Struct Keranjang Belanja menggunakan MAP (Dynamic collection)
type Cart struct {
    CustomerID string
    Items      map[string]float64 // Map: [Nama Produk] → [Harga]
}

// Constructor Cart
```

```

func NewCart(customerID string) *Cart {
    return &Cart{
        CustomerID: customerID,
        Items:      make(map[string]float64),
    }
}

// Method Tambah Item ke Map
func (c *Cart) AddItem(name string, price float64) {
    c.Items[name] = price
}

// Method hitung total & lakukan checkout dengan payment gateway interface
func (c *Cart) Checkout(processor PaymentProcessor) error {
    if len(c.Items) == 0 {
        return errors.New("checkout gagal: keranjang kosong")
    }

    var total float64
    fmt.Println("\nRincian Keranjang Belanja:")
    for item, price := range c.Items {
        fmt.Printf("- %-20s: Rp %12.2f\n", item, price)
        total += price
    }
    fmt.Printf("Total Tagihan          : Rp %12.2f\n", total)
    fmt.Printf("Metode Pembayaran         : %s\n", processor.GetMethodName())

    // Panggil interface method
    err := processor.Pay(total)
    if err != nil {
        return fmt.Errorf("pembayaran via %s gagal: %w", processor.GetMethodName(), err)
    }

    // Transaksi sukses, kosongkan keranjang belanja
    c.Items = make(map[string]float64)
    return nil
}

func main() {
    // Inisialisasi keranjang belanja menggunakan map
    myCart := NewCart("CUST-DIKA-01")
    myCart.AddItem("Keyboard Mechanical", 1200000)
    myCart.AddItem("Mouse Wireless Office", 450000)

    fmt.Println("=====")
    fmt.Println("          APLIKASI CHECKOUT MULTI-GATEWAY          ")
    fmt.Println("=====")

    // Uji 1: Sukses Bayar dengan QRIS
    qris := QrisPayment{QrisID: "QRIS-SHP-091"}
    errQris := myCart.Checkout(qris)
    if errQris != nil {
        fmt.Println("Transaksi Gagal:", errQris)
    }
}

```



```

    } else {
        fmt.Println("Status Transaksi      : SUKSES!")
    }

    // Uji 2: Gagal Bayar karena data invalid (Nomor Kartu Kredit < 16 digit)
    myCart.AddItem("Monitor UltraWide", 4500000) // belanja lagi
    ccInvalid := CreditCard{CardNumber: "1234"}
    errCC := myCart.Checkout(ccInvalid)
    if errCC != nil {
        fmt.Println("\n[LOG KEAMANAN TRANSAKSI ERROR]")
        fmt.Println("Detail Kesalahan:", errCC)
    }
}

```

Sesi 5: Soal Latihan Praktik E-Commerce (Mandiri)

Selesaikan dua latihan pemrograman e-commerce tingkat lanjut di bawah ini untuk memperkuat pemahaman Anda mengenai integrasi Map dinamis dan Interface abstraksi.

Latihan 1: Sistem Validasi Kupon Belanja dengan Map & Sentinel Error

Deskripsi Soal: Buatlah sebuah sistem pencatatan kupon promo e-commerce menggunakan struktur data Map untuk menyimpan data kupon secara dinamis.

Kriteria Implementasi:

- Definisikan sebuah tipe kustom `DiscountRate` berbasis `float64`.
- Buat struct `DiscountManager` yang menampung data map internal:
`map[string]DiscountRate` (Kunci: Kode Kupon, Nilai: Persentase Potongan).
- Definisikan dua **Sentinel Error**:
 - `ErrCouponNotFound` : Error jika kode kupon tidak terdaftar di dalam map.
 - `ErrRateInvalid` : Error jika pendaftaran persentase diskon kurang dari 0% atau lebih dari 100% (nilai `DiscountRate` harus antara `0.0` sampai `1.0`).
- Implementasikan method berikut pada `DiscountManager` :
 - `RegisterCoupon(code string, rate DiscountRate) error` : Memasukkan kode kupon baru (simpan sebagai string uppercase) ke dalam map. Lakukan validasi `rate`. Jika melanggar, kembalikan `ErrRateInvalid`.
 - `CalculateDiscount(code string, originalPrice float64) (float64, error)` : Mencari kupon di dalam map (gunakan *comma ok*). Jika kupon tidak ada, kembalikan `ErrCouponNotFound`. Jika ada, hitung nilai nominal potongan harganya.
- Ujilah sistem di atas di fungsi `main()` dengan mendaftarkan kupon `"SAVE20"` (20%), `"MEGA80"` (80%), mencoba mendaftarkan kupon salah (diskon 150%), serta mencoba kalkulasi harga barang Rp 500.000 menggunakan kupon valid dan kupon tidak terdaftar.

Latihan 2: Sistem Notifikasi Penyiaran E-Commerce (Slice of Interfaces)

Deskripsi Soal: Marketplace membutuhkan modul broadcast notifikasi dinamis yang bisa mengirimkan pesan kampanye promo ke berbagai saluran (Email, SMS, Push Notification) secara bersamaan menggunakan slice dari interface.

Kriteria Implementasi:

1. Buat interface `NotificationChannel` dengan kontrak method:
 - `Send(userID string, message string) error`
 - `GetChannelName() string`
2. Buat tiga concrete struct yang mengimplementasikan interface tersebut secara implisit:
 - `EmailNotifier` (menyimpan field `EmailAddress string`)
 - `SmsNotifier` (menyimpan field `PhoneNumber string`)
 - `PushNotifier` (menyimpan field `DeviceToken string`)
3. Buat struct `NotificationDispatcher` yang memiliki slice kumpulan saluran aktif:
`[]NotificationChannel` (Slice of Interfaces).
4. Implementasikan method berikut pada `NotificationDispatcher` :
 - `AddChannel(channel NotificationChannel)` : Memasukkan saluran notifikasi baru ke dalam slice.
 - `BroadcastNotification(userID string, msg string)` : Lakukan iterasi menggunakan `for range` pada slice saluran aktif. Panggil method `Send` dari tiap saluran. Jika salah satu saluran mengembalikan error, cetak pesan kegagalan tersebut sebagai peringatan, tetapi **tidak boleh menghentikan** proses pengiriman notifikasi pada saluran aktif lainnya di dalam perulangan.
5. Di dalam fungsi `main()` , lakukan setup dispatcher. Tambahkan satu email notifier, satu SMS notifier, dan satu push notifier (simulasikan salah satunya gagal, misalnya karena format token kosong). Jalankan fungsi broadcast promo untuk melihat alur pengiriman tetap berjalan sampai selesai.

Sesi 6: Soal Ujian Akhir Modul (Studi Kasus HRIS)

Instruksi Ujian: Selesaikan studi kasus sistem HRIS (Human Resource Information System) di bawah ini secara mandiri. Kerjakan pada berkas `main.go` terpisah di komputer Anda.

Studi Kasus: Sistem Manajemen Kepegawaian & Payroll Terintegrasi

Sebuah startup membutuhkan modul backend *core* untuk mengelola data kepegawaian (HRIS) dan menghitung anggaran pengeluaran gaji bulanan bersih perusahaan berdasarkan skema kerja karyawan yang bervariasi.

Kriteria Spesifikasi Teknis:

1. Interface `PayrollCalculator` : Mendefinisikan kontrak metode:

- `CalculateSalary() (float64, error)` : Menghitung total gaji bulanan bersih.
- `GetEmployeeType() string` : Mengembalikan tipe karyawan (misal: "FullTime", "Contract", atau "Freelance").

2. Concrete Structs (Mengimplementasikan `PayrollCalculator` secara implisit):

- `FullTimeEmployee` : Memiliki field `BaseSalary float64` , `Allowance float64` , dan `TaxRate float64` (potongan pajak, contoh: `0.05` untuk 5%). *Rumus Gaji: $(BaseSalary + Allowance) * (1.0 - TaxRate)$* .
- `ContractEmployee` : Memiliki field `MonthlyRate float64` dan `PerformanceBonus float64` . *Rumus Gaji: $MonthlyRate + PerformanceBonus$* .
- `Freelancer` : Memiliki field `HourlyRate float64` dan `HoursWorked int` . *Rumus Gaji: $HourlyRate * float64(HoursWorked)$* .

3. Dynamic Collection (Map): Buat struct `HRIS` yang menampung data map internal secara dinamis:

- `Employees map[string]string` : Menyimpan pemetaan `[EmployeeID] → Nama Karyawan` .
- `Payrolls map[string]PayrollCalculator` : Menyimpan pemetaan `[EmployeeID] → Kontrak Penggajian (Interface)` .

4. Method pada Struct `HRIS` (Gunakan Pointer Receiver):

- `RegisterEmployee(id string, name string, payroll PayrollCalculator) error` : Mendaftarkan karyawan baru ke dalam kedua map di atas. Lakukan validasi:
 - Jika ID sudah terdaftar di map, kembalikan error: `"karyawan dengan ID tersebut sudah terdaftar"` .
 - Jika nama karyawan kosong, kembalikan error: `"nama karyawan tidak boleh kosong"` .
 - Jika salah satu nilai keuangan (gaji pokok, rate per jam, dll.) bernilai negatif, kembalikan error.
- `CalculateTotalPayout() float64` : Menghitung total anggaran gaji seluruh karyawan aktif yang terdaftar dengan mengiterasi map menggunakan `for range` .
- `PrintPayrollReport()` : Mencetak slip laporan keuangan penggajian secara rinci. Gunakan **Type Switch** atau **Type Assertion** untuk mendeteksi tipe konkret karyawan dan mencetak detail parameter spesifiknya (contoh: untuk `Freelancer` cetak jumlah jam kerjanya, untuk `FullTimeEmployee` cetak nilai pajaknya).

Petunjuk Pengumpulan Tugas & Integrasi LMS:

1. **Langkah 1 (Repository GitHub)**: Inisialisasi proyek lokal Go Anda, buat berkas kode solusi `main.go` . Buatlah repositori publik baru di akun GitHub pribadi Anda dengan nama: `go-hris-payroll-system` .

2. **Langkah 2 (Git Push):** Lakukan commit kode pengerjaan Anda secara berkala, lalu unggah (*push*) kode solusi tersebut ke repositori GitHub yang telah Anda buat.
3. **Langkah 3 (Pengumpulan LMS):** Salin tautan/URL repositori GitHub Anda (contoh: `https://github.com/username/go-hris-payroll-system`) dan kumpulkan ke kolom tugas yang tersedia di platform **Learning Management System (LMS)** kelas Anda sebelum batas waktu berakhir.